

2.7 - HashMaps

The second major Java data structure is the HashMap. As you probably recall, HashMaps differ from ArrayLists in that you can access items in a HashMap (also called a “map” for short) by a key rather than a position. Later in this book you will see that there are many ways to implement a HashMap. The thing that is most important to notice right now is that the `get` and `put` methods on a HashMap are $O(1)$. Another important HashMap operation is the `containsKey` operation. Checking to see whether a key is in the HashMap or not is also $O(1)$. The efficiency of common HashMap operations is summarized in [Table 2.7.1](#). One important side note on dictionary performance is that the efficiencies we provide in the table are for average performance. In some rare cases the `contains`, `get`, and `set` operations can degenerate into $O(n)$ performance, but we will get into that in Chapter 8 when we talk about the different ways that a dictionary could be implemented.

Table 2.7.1. Big O Efficiency of Java HashMap Operations

Operation	Big O Efficiency
<code>putAll</code> (copy to another HashMap)	$O(n)$
<code>get</code>	$O(1)$
<code>put</code> (set item)	$O(1)$
<code>remove</code>	$O(1)$
<code>containsKey</code>	$O(1)$
iteration	$O(n)$

For our last performance experiment, we will compare the performance of the `contains` method for ArrayLists and the `containsKey` operation for HashMaps. In the process we will confirm that the operation for ArrayLists is $O(n)$ and the operation for HashMaps is $O(1)$. The experiment we will use to compare the two is as follows: we’ll make a list with a range of numbers in it, then we will pick numbers at random and check to see if the numbers are in the list. If our performance tables are correct, the bigger the list, the longer it should take to determine if any one number is contained in the list.

We will repeat the same experiment for a HashMap that contains numbers as the keys. In this experiment we should see that determining whether or not a number is in the HashMap is not only much faster, but the time it takes to check should remain constant even as the HashMap grows larger.

[Listing 2.7.2](#) implements this comparison. Notice that we are performing exactly the same operation, even though they have different names:

`list.contains(number)` and `map.containsKey(number)`.

Listing 2.7.2.

Save & Run



Original - 1 of 1

Show CodeLens

Reformat

```
1 import java.util.ArrayList;
2 import java.util.HashMap;
3 import java.util.Random;
4
5 public class ListVsMap {
6
7     public static void main(String[] args) {
8
9         System.out.println("size      list      map");
10        for (int size = 10_000; size < 1_000_001; size += 20_000) {
11            ArrayList<Integer> list = new ArrayList<>(size);
12            HashMap<Integer, Integer> map = new HashMap<>();
13
14            for (int key = 0; key < size; key++) {
15                list.add(key);
16                map.put(key, key); // key and value are the same
17            }
18            System.gc();
19            Random generator = new Random();
20
21            long startListTime = System.nanoTime();
22            for (int i = 0; i < 1000; i++) {
23                int lookFor = generator.nextInt(size);
24                boolean found = list.contains(lookFor);
25            }
26            long elapsedListTime = System.nanoTime() - startListTime;
27
28            long startMapTime = System.nanoTime();
29            for (int i = 0; i < 1000; i++) {
30                Integer lookFor = generator.nextInt(size);
```

Figure 2.7.3 summarizes the results of running Listing 2.7.2. You can see that the dictionary is consistently faster. For the smallest size of 10,000 elements, a HashMap is 58.2 times faster than an ArrayList. For the largest size of 990,000 elements the HashMap is 3,972 times faster! You can also see that the time it takes for the `contains` method on the ArrayList grows linearly with the size of the list. This verifies the assertion that the `contains` method on a list is $O(n)$. It can also be seen that the time for the `containsKey` method on a HashMap is constant even as the dictionary size grows.

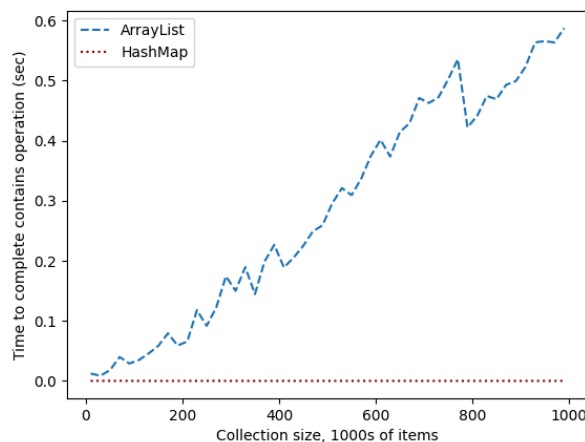


Figure 2.7.3. Comparing Time to Find Elements for Java ArrayList and HashMap

Self Check

1.

Q-1: Which of the ArrayList operations shown below is not $O(1)$?

Choose one

- ☒ A. `myList.remove(0)`
- ☐ B. `myList.remove(myList.size() - 1)`
- ☐ C. `myList.add()`
- ☐ D. `myList.get(10)`
- ☐ E. all of the above are $O(1)$

Check Me

Compare me

2.

Q-2: Which of the HashMap operations shown below is $O(1)$?

Choose one

- ☐ A. `myMap.containsKey("x")`
- ☐ B. `myMap.remove("x")`
- ☐ C. `myMap.put("x", 22)`
- ☒ D. all of the above are $O(1)$

Check Me

Compare me

You have attempted 3 of 4 activities on this page.

EthicalAds

ADS BY ETHICAL



Ultimate BIN Database

Fresh BIN Database of
6-11 digits. Token
support. **Get data**

www.binbase.com

Ads by EthicalAds